Corner Detection & Foreground-Background Segmentation

What is a Corner?

A corner is a point where two or more edges meet, representing a location with significant intensity variation in multiple directions.

Intuition:

- Edges have strong gradients in one direction
- Corners have strong gradients in multiple directions
- Flat regions have no significant gradients

Why detect corners?

- Robust features for tracking and matching
- Invariant to small translations
- Better localized than edges
- Foundation for SIFT, SURF, ORB descriptors

Harris Corner Detector: Motivation

The Window Shifting Idea

Consider a small window around a pixel. As we shift this window in different directions:

- **Flat region**: No change in any direction
- **Edge**: No change along the edge, large change perpendicular to edge
- **Corner**: Large change in all directions

We formalize this by measuring the intensity change when shifting the window by $(\Delta x, \Delta y)$:

$$E(\Delta x, \Delta y) = \sum_{(x,y) \in W} [I(x + \Delta x, y + \Delta y) - I(x, y)]^2$$

where W is the window and I is the image intensity.

Harris Corner Detector: Mathematical Derivation

Using Taylor expansion: $I(x + \Delta x, y + \Delta y) \approx I(x, y) + I_x \Delta x + I_y \Delta y$

This gives us:

$$E(\Delta x, \Delta y) \approx \sum_W [I_x \Delta x + I_y \Delta y]^2$$

In matrix form:

$$E(\Delta x, \Delta y) \approx \begin{bmatrix} \Delta x & \Delta y \end{bmatrix} M \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix}$$

where the structure tensor M is:

$$M = \sum_W \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

Harris Corner Detector: Corner Response

The eigenvalues λ_1, λ_2 of M describe the intensity change directions:

- Both eigenvalues small: flat region
- One large, one small: edge
- Both large: corner

Harris Corner Response Function:

$$R = \det(M) - k \cdot \text{trace}(M)^2$$

$$R = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2$$

where k is typically 0.04 to 0.06.

Corner detection:

- $R > 0$ and large: corner
- $R < 0$: edge
- $|R|$ small: flat region

Harris Corner Detector: Algorithm Steps

Implementation Pipeline:

1. Compute image gradients I_x and I_y (using Sobel or similar)
2. Compute products of derivatives: I_x^2, I_y^2, $I_x I_y$
3. Apply Gaussian smoothing to each product to get M
4. Compute corner response: $R = \det(M) - k \cdot \text{trace}(M)^2$
5. Apply threshold to R
6. Apply non-maximum suppression to get distinct corners

The Gaussian smoothing defines the neighborhood size and reduces noise sensitivity.

Harris Corner Implementation

OpenCV Implementation:

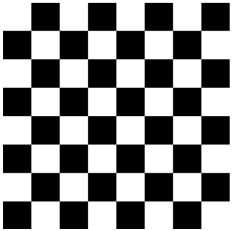
```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray = np.float32(gray)

harris = cv2.cornerHarris(gray, blockSize=2,
                          ksize=3, k=0.04)

img[harris > 0.01 * harris.max()] = [0, 0, 255]
```

Parameters:

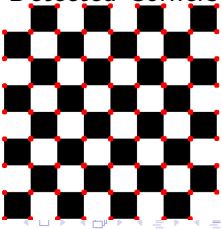
- `blockSize`: Neighborhood size for computing M
- `ksize`: Aperture for Sobel gradient
- `k`: Harris detector free parameter (0.04-0.06)

Harris Corner Detection

Original Image



Detected Corners



Shi-Tomasi Corner Detector

Improvement over Harris

Instead of Harris's response function, Shi-Tomasi uses:

$$R = \min(\lambda_1, \lambda_2)$$

Advantages:

- More stable corner selection
- Better for tracking applications
- Used in optical flow algorithms
- Avoids tuning parameter k

A corner is detected if $R >$ threshold.

OpenCV Implementation:

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

corners = cv2.goodFeaturesToTrack(
  gray, maxCorners=100, qualityLevel=0.01,
  minDistance=10)

for corner in corners:
  x, y = corner.ravel()
  cv2.circle(img, (x, y), 3, (0, 0, 255), -1)
```

Parameters:

- `maxCorners`: Maximum number of corners
- `qualityLevel`: Minimum corner quality
- `minDistance`: Minimum distance between corners

FAST Corner Detector

Features from Accelerated Segment Test

FAST is designed for real-time applications:

1. Select a pixel p with intensity I_p
2. Consider circle of 16 pixels around p
3. A corner if n contiguous pixels are all brighter or all darker than $I_p + t$ or $I_p - t$
4. Typically $n = 12$ (FAST-12) and threshold t is chosen empirically

Advantages:

- Extremely fast (no gradients, no convolutions)
- Good for real-time video processing
- Used in ORB, BRISK feature descriptors

Disadvantages:

- Not rotation invariant
- Requires non-maximum suppression
- Multiple responses near corners

FAST Implementation

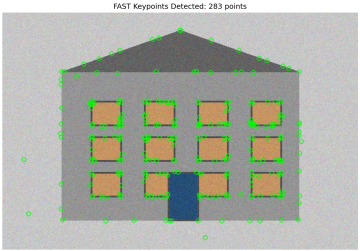
OpenCV Implementation:

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

fast = cv2.FastFeatureDetector_create(
  threshold=10, nonmaxSuppression=True)

keypoints = fast.detect(gray, None)

img_fast = cv2.drawKeypoints(img, keypoints,
  None, color=(0, 0, 255))
```

Algorithm Optimization:

- High-speed test: Check pixels 1, 9, 5, 13 first
- If at least 3 are above/below threshold, check all 16
- Reject as corner otherwise

FAST Corner Detection



Detected 150+ corners in ¡1ms

Image Segmentation: Overview

Goal: Partition an image into meaningful regions

Types of Segmentation:

- **Semantic segmentation**: Classify every pixel (e.g., road, car, person)
- **Instance segmentation**: Separate individual objects
- **Foreground-background segmentation**: Binary classification

Foreground-Background Segmentation

Simplest form: separate objects of interest (foreground) from background.

Applications:

- Object extraction for compositing
- Image editing and manipulation
- Background removal for virtual meetings
- Document scanning and text extraction

Simple Thresholding

Basic Approach: Use intensity to separate foreground and background

For grayscale image $I(x, y)$:

$$B(x, y) = \begin{cases} 1 & \text{if } I(x, y) > T \\ 0 & \text{otherwise} \end{cases}$$

where T is a threshold value and B is the binary output.

Threshold Selection:

- **Manual**: User selects threshold value
- **Automatic**: Algorithm determines optimal threshold
- **Adaptive**: Different thresholds for different regions

Otsu's Thresholding Method

Automatic Threshold Selection

Otsu's method finds the threshold that minimizes intra-class variance (or equivalently, maximizes inter-class variance).

Algorithm:

1. Compute histogram of image intensities
2. For each possible threshold t:
   - Compute class probabilities $w_0(t)$, $w_1(t)$
   - Compute class means $\mu_0(t)$, $\mu_1(t)$
   - Compute inter-class variance
3. Select t that maximizes inter-class variance

Inter-class variance:

$$\sigma_b^2(t) = w_0(t)w_1(t)[\mu_0(t) - \mu_1(t)]^2$$

Assumptions:

- Bimodal intensity distribution
- Equal variances in both classes
- Works best with good contrast

Optimal for images where foreground and background have distinct intensities.

Otsu's Thresholding Implementation

OpenCV Implementation:

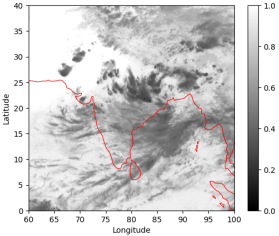
```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

threshold, binary = cv2.threshold(gray, 0, 255,
  cv2.THRESH_BINARY + cv2.THRESH_OTSU)

print(f"Optimal threshold: {threshold}")
```

The threshold value 0 is ignored; Otsu's method automatically finds the optimal value.

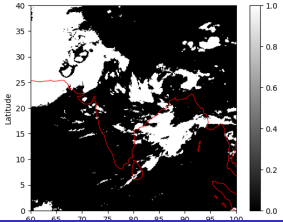
Variations:

- `THRESH_BINARY`: Foreground white, background black
- `THRESH_BINARY_INV`: Inverted

Otsu's Thresholding
Original



Binary Result



Adaptive Thresholding

Problem with Global Thresholding

A single threshold fails when:

- Illumination varies across the image
- Foreground/background intensities change locally
- Shadows or gradients present

Adaptive Solution:

Compute different thresholds for different regions:

$$T(x, y) = \text{mean}(I_{\text{local}}) - C$$

where I_{local} is the neighborhood around (x, y) and C is a constant.

Methods:

- **Mean**: Threshold = mean of neighborhood minus C
- **Gaussian**: Threshold = Gaussian-weighted sum minus C

```
gray = cv2.cvtColor(img,
cv2.COLOR_BGR2GRAY)
```

```
adaptive = cv2.adaptiveThreshold(gray,
  255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
  cv2.THRESH_BINARY, blockSize=11, C=2)
```

Parameters:

- `blockSize`: Neighborhood size (odd number)
- `C`: Constant subtracted from mean

K-Means Clustering for Segmentation

Color-Based Segmentation

K-means groups pixels by color similarity:

1. Represent each pixel as a feature vector (e.g., RGB or LAB)
2. Initialize K cluster centers
3. Assign each pixel to nearest cluster
4. Update cluster centers
5. Repeat until convergence

Objective:

$$\min \sum_{k=1}^{K} \sum_{x \in C_k} \|x - \mu_k\|^2$$

where C_k is cluster k and μ_k is its center.

For Segmentation:

- $K = 2$: Binary foreground-background
- $K > 2$: Multi-region segmentation
- Feature space: color, texture, position

```
pixels = img.reshape((-1,
3)).astype(np.float32)

criteria = (cv2.TERM_CRITERIA_EPS +
  cv2.TERM_CRITERIA_MAX_ITER, 100, 0.2)

K = 2
ret, label, center = cv2.kmeans(pixels,
  K, None, criteria, 10,
  cv2.KMEANS_RANDOM_CENTERS)

center = np.uint8(center)
result = center[label.flatten()]
result = result.reshape(img.shape)
```

Mean Shift Segmentation

Mode-Seeking Algorithm

Mean shift finds modes (peaks) in the feature space density:

- Each pixel starts at its own position in feature space
- Iteratively shift toward the mean of neighbors
- Converge to local density maxima
- Pixels converging to same mode form a segment

Update Rule:

$$x_{t+1} = \frac{\sum_{x_i \in N(x_t)} K(x_i - x_t) \cdot x_i}{\sum_{x_i \in N(x_t)} K(x_i - x_t)}$$

where K is a kernel function and $N(x_t)$ is the neighborhood.

Advantages:

- No need to specify number of segments
- Preserves edges well
- Robust to initialization

```
shifted = cv2.pyrMeanShiftFiltering(img,
  sp=21, sr=51)

gray = cv2.cvtColor(shifted,
  cv2.COLOR_BGR2GRAY)
ret, thresh = cv2.threshold(gray, 0, 255,
  cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

Parameters:

- `sp`: Spatial radius
- `sr`: Color radius